Assignment 2: In-order multi-issue and a branch unit for a Tomasulo simulator with a reorder buffer

COSC 6385 Computer Architecture, Fall 2026

Bryan E. Tuck
University of Houston
https://github.com/Bryan-Tuck/cosc6385-hw2-tomasulo-simulator

October 2026

1 Introduction

A Tomasulo machine with a reorder buffer (ROB) renames registers at issue, lets instructions execute as soon as their operands arrive, and commits them in program order [3, 4]. The starter code for this assignment [2] issues one instruction per cycle and stops fetching at every branch until the branch has executed. The assignment asks for in-order multi-issue, up to `issue_width` instructions (1 to 4, default 1) entering the ROB and the reservation stations in the same cycle, with the width set by an `Issue width = N` line in the input file. The handout also describes the branch unit the machine should have: a one-bit predictor with an 8-entry branch target buffer (BTB), prediction at ISSUE, resolution at the end of EX, and recovery from a misprediction detected in cycle n with correct fetch resuming in cycle $n + 2$. The starter has no predictor or BTB (its README says so), so I added them.

With more than one instruction issuing per cycle, the instructions in a group are renamed one after another, so a later instruction reads the ROB tag an earlier one just wrote into the register alias table (RAT) instead of the stale value behind it [4, slide 13, "Intra-group dependencies"]. The group stops at the first instruction that finds no free ROB entry, reservation station or load/store queue slot, and everything behind it waits. Several instructions can become ready in the same cycle; the single CDB broadcasts one result per cycle, while the commit stage may retire a whole group. Instructions also issue past an unresolved branch, so a misprediction leaves wrong-path instructions in the ROB, the reservation stations, the functional units, the result queue and the load/store queue, and recovery has to remove all of them and restore the RAT.

I added the multi-issue logic to `issue.py`, a parser (`parse_input.py`) for the handout's input format, the branch unit (`branch.py`), and the register and memory dump of the handout's output format. The commit stage retires up to `issue_width` ready instructions per cycle, the commit width of the course's sample report and two-issue example [1, 5], and there is one common data bus (CDB), as the handout specifies. On eight independent instructions the cycle count falls from 20 at width 1 to 14 at width 4; on a six-instruction RAW chain it stays at 26 at every width; on every test the final registers and memory are the same at every width and equal to the values computed by hand.

2 Pipeline model

The machine has the five stages of the handout: ISSUE (fetch, decode, rename), EX, MEM, WB and COMMIT. Table 1 lists the timing rules the simulator follows, including the changes to the starter described in Section 3. As in the course's walk-through examples [5], an instruction issued in cycle c executes from $c+1$, writes back the cycle after it finishes, and commits the cycle after it writes back. The two-issue example there broadcasts two results per cycle; the handout specifies one CDB, so this simulator broadcasts one.

Table 1: Timing rules. c is the cycle in which the event in the left column happens.

Event	Rule
Issue in cycle c	Up to `issue_width` instructions per cycle in program order; EX can start in $c+1$. A reservation station is free again the cycle after its instruction starts EX.
Operands arrive on the CDB in c	EX can start in $c+1$; the oldest ready instruction in a reservation station goes first.
Integer adder, address adder	Unpipelined: a unit that finishes in c takes a new instruction in $c+1$.
FP adder, FP multiplier	Pipelined: one new instruction per cycle per unit.
Result produced in c	Broadcast on the single CDB in $c+1$ at the earliest; results queue in completion order, and results produced in the same cycle queue in stage order (EX before MEM).
Load address known in c	Memory access from $c+1$ if every older store's address is known and none matches; store-to-load forwarding takes one MEM cycle ($c+1$) when the nearest older store with the same address has its data.
Load/store queue	A load leaves the queue when its data returns, from memory or from a store; a store leaves when its memory write ends. The slot can be reused from the next cycle.
Memory port	Single, non-pipelined; a load that can use the port takes it before a store that is ready to commit in the same cycle, and a committing store waits for a load that already holds it.
Commit	Up to `issue_width` ROB-head entries per cycle, in order, stopping at the first that is not ready. An instruction commits at the earliest in the cycle after its write-back (a branch, after it resolves); a store commits once its address and data are known and the port is free, and starts its memory write in its commit cycle. A ROB entry can be reused in its commit cycle.
Branch issued in c	Predicted from the BTB at issue; fetch continues from the predicted target in c (fall-through) or $c+1$ (taken). Resolved at the end of EX.
Misprediction detected in n	Wrong-path instructions are squashed and the RAT restored; no issue in $n+1$; issue from the correct target in $n+2$.

3 What I added

3.1 Input file and output

The original `main.py` hard-coded the reservation-station counts, latencies, ROB size and the initial registers and memory, and read a bare list of instructions from `code.in`. `parse_input.py` reads

the handout's format instead: the functional-unit table, `ROB entries`, `Issue width`, register and memory initial values, and the program. Matching is case-insensitive, commas are optional, a line starting with `#` or `//` is a comment, and a missing configuration line keeps the starter's hard-coded value (the issue width defaults to 1). The decoder in `init.py` replaces the starter's `split(' ')` on raw text with an `instruction` record, which also fixes two-digit register numbers such as `R10`, and accepts `Beq`, which the starter lacked. `main.py` takes the input path on the command line and still defaults to `code.in`, which now holds the original program with the formerly hard-coded configuration; `--issue-width N` overrides the file's width, so each test file runs at all four widths without a copy per width. All simulator state now lives in one `machine` object (`machine.py`) built from the parsed configuration, and each stage function takes it as its argument instead of the starter's long argument lists. After the cycle table, which keeps the starter's format, `main.py` prints a second table with the squashed wrong-path instructions, then the `REGISTER VALUES` and `NON-ZERO MEMORY VALUES` sections of the handout. Two summary lines follow with the issue width, the number of committed instructions, the cycle count, the IPC, and the branch and misprediction counts. The cycle count is the last cycle in which anything happened; for a program ending in a store, that is the last cycle of the memory write.

3.2 Multi-issue

`issue()` in `issue.py` loops up to `issue_width` times per cycle. Each iteration takes the instruction at the program counter, checks that the ROB has a free entry and that its reservation station (or the load/store queue) has a free slot, and if so allocates the ROB entry, reads both operands through the RAT, writes the entry into the station, and finally points the RAT entry of the destination register at the new ROB tag. Because the RAT is updated before the next instruction of the group reads it, an intra-group RAW dependence is renamed correctly. In `t2_raw_chain` at width 4, the second `Add.d F1 F1 F2` issued in cycle 1 waits on the tag `ROB0` of the first rather than on the initial value of `F1`, and the six instructions still take 26 cycles. The loop stops at the first instruction that finds no space, so the group is in order and nothing behind a stalled instruction issues ahead of it; the `t3_structural` test exercises both a full reservation station and a full ROB. A branch is predicted when it issues (Section 3.4); a predicted-taken branch ends the group, because fetch has to be redirected, while a predicted-not-taken one lets the group continue with the fall-through instruction.

Two changes elsewhere were needed before a width above 1 gave correct results. First, the starter's CDB broadcast wrote the result value into the RAT unconditionally. With two writers of the same register in flight at once, which multi-issue makes common, the older writer's broadcast overwrote the younger writer's ROB tag, and a later reader picked up a stale value. `wb.py` now writes the RAT only when the RAT entry still holds the broadcasting tag, which is the rule on slide 34 of Lecture 8 [3]. Second, the starter dispatched the lowest-numbered ready entry of a reservation station, which is not always the oldest. `exe.py` now picks the oldest ready entry by ROB order, and only among instructions issued in an earlier cycle, so EX starts at $c + 1$ as in Table 1. The same file limits the unpipelined integer adder to one instruction in flight and lets the pipelined FP units accept one new instruction per cycle; the starter advanced every entry of every unit each cycle, which pipelines the integer adder when its latency is above 1.

3.3 Commit and the CDB

The commit stage retires up to `issue_width` entries per cycle from the ROB head, in order, and stops at the first entry that is not ready: an ALU or load result must have been broadcast, a store

must have its address, its data and a free memory port, and a branch must have resolved. The commit width follows the issue width, as in the course's sample report and two-issue example [1, 5]; it is not a separate setting. If commit stayed at one per cycle, the ROB would fill behind any long-latency instruction and issue would then run at one per cycle too. A mispredicted branch never brings wrong-path instructions into a commit group, because the squash happens in the EX stage of the cycle in which the branch resolves, before the commit stage runs, and the branch is marked ready only then. By the time it reaches the head, everything younger than it on the wrong path is gone. Stores still serialize on the single memory port, so two stores never commit in the same cycle. The sample report also broadcasts `issue_width` results per cycle; the handout specifies one CDB, so I kept one. Results enter a queue in the cycle they are produced and the oldest queued result is broadcast each cycle. Instructions that write a register therefore pass the CDB one per cycle, and no test reaches an IPC of 1 (Section 5).

3.4 Branch unit

`branch.py` holds the predictor, the BTB and the recovery. The BTB has eight entries indexed by the low three bits of the branch's word address (its index in the program; the byte PC is four times that). Each entry stores the branch's PC as a tag, the target it last jumped to, and one bit with its last outcome, which is the one-bit predictor. When a branch issues, `issue()` looks it up. A valid entry with a matching tag and a set bit predicts taken with the stored target; anything else predicts not-taken. The predicted next PC is recorded in the ROB entry together with a checkpoint of both RATs, and fetch continues along the predicted path. The branch itself goes into an integer-adder reservation station like an `Add`, with the two registers to compare as its operands.

At the end of EX, `resolve_branch` compares the operands (`Beq` or `Bne`), computes the real target, updates the BTB entry with the outcome and target, and compares them with the prediction. If both agree the branch is marked ready and commits in order later. If not, the misprediction is handled in the same cycle n in the three steps the handout lists. First, every ROB entry younger than the branch is popped from the tail, and with it everything that belongs to those instructions. Their reservation-station entries are freed, which clears the wait-for tags, and their entries in the functional units, the results queue, the load/store queue, the address adder and the memory port are dropped. The ROB tail pointer moves back to the slot after the branch, so tags are not duplicated. A store that has already committed is never squashed. It keeps its queue entry and finishes its memory write, even when the ROB tag it had now belongs to a wrong-path instruction. Second, the RAT is restored from the checkpoint taken at issue. A checkpointed entry may hold a tag of an instruction that broadcast after the checkpoint (its value is taken from the ROB entry) or that has already committed (the architectural register is taken), so the restored RAT is consistent with the machine's state in cycle n rather than in the issue cycle. Third, the program counter is set to the correct target and issue is blocked for cycle $n + 1$, so the first correct-path instruction issues in $n + 2$. Squashed instructions appear in a second output table with the cycle in which they were squashed. `t5_loop` (Section 4) checks the recovery: after two mispredictions, every register and memory value is correct.

The branch offset is in bytes and is added to PC + 4, the convention of the starter's test case: its `Bne R1 R0 -12`, the eighth instruction, goes to the sixth. The handout's formula, `PC + 4 + (offset << 2)`, would send that branch to byte address −16, before the start of the program, so I kept the starter's reading, which the provided `code.in` and `test_case.txt` depend on, and wrote my own loops with multiples of 4. A taken branch whose offset is not a multiple of 4 stops the simulation with an error.

3.5 Loads and stores

The load/store queue keeps the starter's structure (a FIFO with one address adder and one memory port) with three corrections that the forwarding test exposed. The starter sent a store to memory as soon as it reached the head of the queue with its data, before its turn to commit, because its commit check was a function object that was never called; with speculation this would let a wrong-path store write memory. Stores now go to memory from `commit.py` when they are the ROB head, as the handout requires, and the queue entry stays until the write completes so that a younger load to the same address can still forward from it. The forwarding check looked only at the nearest older store, whatever its address; it now walks the older stores from nearest to farthest, forwards from the first one with the same address, waits at a store whose address or data is unknown, and otherwise lets the load read memory. Forwarding takes one MEM cycle and a memory access starts the cycle after the address is known, so both paths start MEM in the same cycle. Finally, the "already has data" test that gated loads read the queue head instead of the load's own entry, which blocked every load behind a store that was holding data. A load also keeps its queue entry until its data returns, from memory or from a store, as the handout specifies; the starter freed the entry as soon as the load entered the memory port.

4 Test cases

Each test is an input file in `tests/`; Table 2 lists what each one exercises and the final state it must produce, which I computed by hand from the program. Every test runs at widths 1, 2, 3 and 4 with `--issue-width`, and the outputs are in `results/`, one file per test and width; Appendix A has all 32 cycle tables. All of them use one unit of each type, a 1-cycle integer adder, a 3-cycle FP adder, a 10-cycle FP multiplier (20 in the handout's sample) and a 4-cycle memory, except `original_code_in`, which keeps the starter's configuration (4-cycle FP adder, 15-cycle multiplier, 5-cycle memory, 64-entry ROB).

Hand check of a multi-issue table. Table 3 is `t1_independent` at width 4, checked against the rules of Table 1. In cycle 1 the first four instructions issue (the load/store queue has three slots, the FP adder three stations, the multiplier two). The remaining four issue in cycle 2 (the integer adder has four stations). `Ld F5` computes its address in cycle 2 on the single address adder and holds the memory port in cycles 3 to 6; `Ld F6` gets the adder in cycle 3 and the port in cycles 7 to 10. `Add.d F7` and `Mult.d F8` start in cycle 2 (issued in cycle 1). Of the integer instructions issued in cycle 2, `Add R5`, `Sub R6` and `Addi R7` take the unpipelined adder one per cycle in cycles 3, 4 and 5; `Sub.d F9` enters the pipelined FP adder in cycle 3 behind `Add.d F7`. The WB column is the CDB queue: `Add R5` finishes in 3 and broadcasts in 4, `Add.d F7` finishes in 4 and broadcasts in 5, `Sub R6` finishes in 4 but waits until 6, `Sub.d F9` (done in 5) gets 7, `Addi R7` (done in 5) gets 8, `Ld F5` (data in 6) gets 9, `Ld F6` (data in 10) gets 11 and `Mult.d F8` (done in 11) gets 12. Commit is in order and up to four per cycle: `Ld F5` in 10; `Ld F6` (WB 11) and `Add.d F7` (ready since cycle 5) together in 12; `Mult.d F8` (WB 12), `Add R5`, `Sub R6` and `Sub.d F9` together in 13, a full group of four; `Addi R7` in 14. Width 1 (`results/t1_independent.w1.txt`) issues the same instructions over cycles 1 to 8 and finishes in 20. Of the six cycles saved, three come from issue. `Mult.d F8` issues in cycle 1 instead of 4, so it writes back in 12 instead of 15. The other three come from commit. At width 1 the four instructions behind `Mult.d F8` commit one per cycle in cycles 17 to 20, while at width 4 three of them commit with it in cycle 13 and the last in 14.

Table 2: Test programs. The last column is the hand-computed final state that every width must produce.

Test	What it exercises	Final state
`original_code_in`	The starter's program (`code.in`, same as `test_case.txt`): a loop whose `Bne R1 R0 -12` jumps back two instructions, so the body repeats only the two `Addi`s. Three branches, two mispredicted (BTB miss on the first, fall-through on the last).	R1=0, R2=20, F2=1, F4=3, F6=7, F20=2, Mem[32]=7 (the values the starter computes)
`spec_sample`	The handout's sample configuration and program (20-cycle multiplier, 128-entry ROB). As printed the loop never ends (R2 stays 20, R3 is 0), so one `Addi R2 R2 -10` counts R2 down and the `Bne` is taken once; both branches are mispredicted.	R2=0, F1=30.1, F4=0 (Mem[18] is 0)
`t1_independent`	Eight instructions with no dependences: two loads and six ALU instructions (`Add`, `Sub`, `Addi`, `Add.d`, `Sub.d`, `Mult.d`) spread over the three ALUs. Shows the gain from issue width and the commit and CDB limits.	F5=10, F6=20, F7=3, F8=12, R5=12, R6=−4, F9=3, R7=112
`t2_raw_chain`	Six `Add.d F1 F1 F2` in a row, each dependent on the previous one (six FP adder stations, so only the dependence limits it).	F1=4.0
`t3_structural`	One FP multiplier station and a 4-entry ROB: the second and third `Mult.d` wait for the station; `Add.d F7` and the three `Addi` wait for ROB entries.	F3=F4=F5=6, F6=F7=5, R1=5, R2=6, R3=7
`t4_forwarding`	Store-to-load forwarding (`Ld F4`, `Ld F6`, `Ld F7`), a load that bypasses an older store to another address (`Ld F5`), and loads that wait for an older store's data (`Ld F4`, `Ld F7`).	F2=1, F3=6, F4=6, F5=2, F6=5, F7=6, Mem[8]=6, Mem[16]=5
`t5_loop`	A four-iteration loop that sums Mem[0..12] into F4 and stores the sum; `Bne R1 R3 -16` is taken three times. The first `Bne` misses in the BTB and the last is predicted taken, so two recoveries; the `Sd` issued down the wrong path must not reach memory.	R1=16, F4=10, Mem[16]=10
`t6_beq_mispredict`	`Beq` taken on a BTB miss (the skipped `Addi R4` issues and is squashed), `Beq` not taken, and `Sub`, `Sub.d`, `Mult.d`.	R4=0, R5=2, F3=1, F4=6, R6=1, R7=3

Hand check of a misprediction. Table 4 is `t5_loop` at width 1. The first `Bne R1 R3 -16` issues in cycle 4 and finds no BTB entry, so it is predicted not-taken and the fall-through `Sd F4 0(R3)` issues in cycle 5 and even computes its address in cycle 6. The branch waits for `Addi R1`, whose result is broadcast in cycle 5, executes in cycle 6 and finds `R1` $= 4 \neq 16$: taken, a misprediction detected in $n = 6$. In that cycle the `Sd` is squashed (first row of the squashed table, cycle 6), its load/store queue entry is dropped, the RAT checkpoint is restored, and the BTB entry for word address 3 is written with "taken, target 0". Nothing issues in cycle 7, and the correct `Ld F2 0(R1)` issues in cycle $n + 2 = 8$. The second `Bne` (issue 11) hits in the BTB, is predicted taken, and the next `Ld F2` issues in cycle 12 with no bubble; the branch resolves in 13 and agrees. The fourth `Bne` (issue 19) is predicted taken but falls through (`R1` $= 16$): detected in cycle 21, the `Ld F2` issued in

20 and the `Add.d` issued in 21 are squashed, and the `Sd F4` that follows the loop issues in cycle 23. The store commits in 31 and writes Mem[16] = 10.0 in cycles 31 to 34, the sum of the four loads. No wrong-path store reached memory, and the correct-path instructions read the right operands from the restored RAT.

Table 3: `t1_independent`, width 4: 14 cycles.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F5 0(R1)`	1	2–2	3–6	9	10
`Ld F6 0(R2)`	1	3–3	7–10	11	12
`Add.d F7 F1 F2`	1	2–4		5	12
`Mult.d F8 F3 F4`	1	2–11		12	13
`Add R5 R1 R2`	2	3–3		4	13
`Sub R6 R3 R4`	2	4–4		6	13
`Sub.d F9 F4 F1`	2	3–5		7	13
`Addi R7 R3 100`	2	5–5		8	14

Table 4: `t5_loop`, width 1: 34 cycles, two mispredictions.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F4 F4 F2`	2	8–10		11	12
`Addi R1 R1 4`	3	4–4		5	13
`Bne R1 R3 -16`	4	6–6			14
`Ld F2 0(R1)`	8	9–9	10–13	14	15
`Add.d F4 F4 F2`	9	15–17		18	19
`Addi R1 R1 4`	10	11–11		12	20
`Bne R1 R3 -16`	11	13–13			21
`Ld F2 0(R1)`	12	13–13	14–17	19	22
`Add.d F4 F4 F2`	13	20–22		23	24
`Addi R1 R1 4`	14	15–15		16	25
`Bne R1 R3 -16`	15	17–17			26
`Ld F2 0(R1)`	16	17–17	18–21	22	27
`Add.d F4 F4 F2`	17	24–26		27	28
`Addi R1 R1 4`	18	19–19		20	29
`Bne R1 R3 -16`	19	21–21			30
`Sd F4 0(R3)`	23	24–24	31–34		31
Squashed (last column: cycle squashed)					
`Sd F4 0(R3)`	5	6–6			6
`Add.d F4 F4 F2`	21				21
`Ld F2 0(R1)`	20	21–21			21

5 Results under each issue width

Table 5 gives the cycle count, IPC, branch count and misprediction count of every test at widths 1 to 4; the final registers and memory are identical at every width for every test and match Table 2. On `original_code_in` they are the values the unmodified starter computes, and the rows before the first branch are identical to the starter's cycle table; from the first branch on the predictor changes the timing (Table 6 in the appendix).

Table 5: Cycles, IPC, branches and mispredictions per test and issue width (commit width = issue width, one CDB). Final registers and memory are the same at every width.

		Cycles				IPC					
Test	Instr.	1	2	3	4	1	2	3	4	Br.	Mispr.
`original_code_in`	15	42	35	34	34	0.36	0.43	0.44	0.44	3	2
`spec_sample`	8	19	15	13	13	0.42	0.53	0.62	0.62	2	2
`t1_independent`	8	20	16	15	14	0.40	0.50	0.53	0.57	0	0
`t2_raw_chain`	6	26	26	26	26	0.23	0.23	0.23	0.23	0	0
`t3_structural`	8	22	21	21	21	0.36	0.38	0.38	0.38	0	0
`t4_forwarding`	9	21	19	19	19	0.43	0.47	0.47	0.47	0	0
`t5_loop`	17	34	31	30	29	0.50	0.55	0.57	0.59	4	2
`t6_beq_mispredict`	7	21	18	17	16	0.33	0.39	0.41	0.44	2	1

`t1_independent` gains the most. It drops from 20 to 16, 15 and 14 cycles as the width grows, because the eight instructions issue in 8, 4, 3 and 2 cycles and the commits at the end go out in groups (hand check above). The RAW chain of `t2_raw_chain` takes 26 cycles at every width, because each `Add.d` waits for the previous broadcast, executes for 3 cycles and broadcasts the cycle after, so the chain costs 4 cycles per link regardless of how early the links issued. `t3_structural` improves by one cycle only, from 22 to 21, because the single multiplier station and the 4-entry ROB block issue most of the time. The one cycle comes from commit. At width 2 `Mult.d F5` and `Add.d F6` commit together in cycle 17, so every later commit moves one cycle earlier. At width 2 `Add.d F6` also issues in cycle 5 instead of 6, but that only moves its own EX and WB, which finish long before it can commit. `t4_forwarding` goes from 21 to 19 cycles. One cycle comes from issue. At widths 2 to 4 `Ld F6` forwards in cycle 8 instead of 11, which leaves the CDB free for `Ld F4` in cycle 13 instead of 14. The other comes from commit. At width 1 the entries ahead of the second store retire one per cycle (`Ld F4`, `Ld F5`, `Addi R3` in cycles 15 to 17) and `Sd F1` starts its write in cycle 18; at width 4 they retire together in cycle 14 and the write starts in cycle 16, as soon as the port is free.

In `t5_loop` the first and last `Bne` are mispredicted and the middle two are predicted correctly; the cycle count goes from 34 to 31, 30 and 29; each iteration's `Bne` depends on its `Addi`, and after each misprediction the pipeline refills. `spec_sample` and `t6_beq_mispredict` show the same pattern (19 to 13 and 21 to 16). On `original_code_in` the simulator takes 42 cycles at width 1 against the starter's 39. The three branches now commit in order and take a commit cycle each at width 1, where the starter dropped a resolved branch from the ROB without one, and that is the whole difference at the back end; at the front end the two mispredictions each cost one more cycle than the starter's stall (fetch resumes in $n + 2$ instead of $n + 1$) and the correctly predicted middle branch saves one, so the final `Add.d F20` issues in cycle 19 instead of 18. At widths 2 to 4 (35, 34, 34 cycles) the branches commit in the same group as their neighbours and the issue stage is no longer the limit; the 15-cycle multiplier on the critical path is. More squashed instructions at the wider widths (4 against 2 on the original program, 6 against 3 on `t5_loop`) are the wrong-path instructions a wider front end fetches before the branch resolves.

6 What works and what does not

The features the handout asks for are implemented and tested: issue widths 1 to 4 from the input file (or the command line); intra-group renaming; in-order stall at a full ROB, reservation station or load/store queue; commit of up to `issue_width` instructions per cycle; one CDB; the one-bit predictor and 8-entry BTB with prediction at issue; misprediction recovery with the $n + 2$ restart, including a wrong-path store that is squashed before it reaches memory; loads and stores with forwarding; `Beq` and `Bne`; the handout's input and output formats. All eight tests give the same final state at every width, and the hand checks above agree with the simulator.

Some choices are not fixed by the handout. The commit width equals the issue width, as in the sample report, and is not a separate parameter. There is one CDB, granted in completion order, so IPC stays below 1 on every test. Branch offsets are byte offsets (Section 3.4). The handout says a load goes to memory when no matching store is found or when older store addresses are still unresolved. Taken literally, a load could then read memory before an older store to the same address writes it, and the handout gives no way to replay the load, so here a load waits until every older store's address is known, as in the starter. A BTB entry is used only when its tag matches, so two branches that share an index evict each other rather than share a prediction, and a wrong-path branch that resolves before the older branch does updates the BTB. The "number of FUs" column

applies to the three ALUs; the memory stays single-ported whatever the load/store unit's count says, and at most one store-to-load forward happens per cycle. Memory is 256 entries indexed by byte address; a load or store moves one value at that address and no alignment is checked. Squashed instructions are listed separately, so the main table is the committed program in order. The sample report [1] broadcasts `issue_width` results per cycle and has no predictor, so its cycle tables are not comparable with these.

References

[1] Computer architecture programming assignment: Tomasulo (sample report). COSC 6385 course materials, University of Houston, 2026.

[2] Weidong Shi. Assignment: Tomasulo algorithm implementation. COSC 6385 Computer Architecture, University of Houston, programming assignment handout, 2026.

[3] Weidong Shi. Lecture 8: Dynamic scheduling. COSC 6385 lecture slides, University of Houston, 2026.

[4] Weidong Shi. Lecture 9: Register rename and ROB. COSC 6385 lecture slides, University of Houston, 2026.

[5] Weidong Shi. OOO examples. COSC 6385 lecture slides (single-issue and two-issue Tomasulo walk-throughs), University of Houston, 2026.

A Cycle tables at every width

These tables show every test at widths 1 to 4, as printed by the simulator (the files in `results/`). Squashed instructions are listed under each table, with the cycle in which they were squashed in the last column.

Table 6: `original_code_in` at widths 1 to 4.

Width 1: 42 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	9	10
`Mult.d F4 F2 F20`	2	10–24		25	26
`Ld F6 0(R2)`	3	4–4	8–12	14	27
`Add.d F6 F4 F6`	4	26–29		30	31
`Sd F6 0(R2)`	5	6–6	32–36		32
`Addi R1 R1 -4`	6	7–7		8	33
`Addi R2 R2 -4`	7	8–8		10	34
`Bne R1 R0 -12`	8	9–9			35
`Addi R1 R1 -4`	11	12–12		13	36
`Addi R2 R2 -4`	12	13–13		15	37
`Bne R1 R0 -12`	13	14–14			38
`Addi R1 R1 -4`	14	15–15		16	39
`Addi R2 R2 -4`	15	16–16		17	40
`Bne R1 R0 -12`	16	17–17			41
`Add.d F20 F2 F2`	19	20–23		24	42
Squashed (last column: cycle squashed)					
`Add.d F20 F2 F2`	9				9
`Addi R1 R1 -4`	17				17

Width 2: 35 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	2	3–3	8–12	14	25
`Add.d F6 F4 F6`	2	25–28		29	30
`Sd F6 0(R2)`	3	4–4	30–34		30
`Addi R1 R1 -4`	3	4–4		5	31
`Addi R2 R2 -4`	4	5–5		6	31
`Bne R1 R0 -12`	4	6–6			32
`Addi R1 R1 -4`	8	9–9		10	32
`Addi R2 R2 -4`	8	10–10		11	33
`Bne R1 R0 -12`	9	11–11			33
`Addi R1 R1 -4`	10	12–12		13	34
`Addi R2 R2 -4`	10	13–13		15	34
`Bne R1 R0 -12`	11	14–14			35
`Add.d F20 F2 F2`	16	17–20		21	35
Squashed (last column: cycle squashed)					
`Add.d F20 F2 F2`	5				6
`Bne R1 R0 -12`	14				14
`Addi R2 R2 -4`	13				14
`Addi R1 R1 -4`	12				14

Width 3: 34 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	1	3–3	8–12	14	25
`Add.d F6 F4 F6`	2	25–28		29	30
`Sd F6 0(R2)`	2	4–4	30–34		30
`Addi R1 R1 -4`	2	3–3		4	30
`Addi R2 R2 -4`	3	4–4		5	31
`Bne R1 R0 -12`	3	5–5			31
`Addi R1 R1 -4`	7	8–8		9	31
`Addi R2 R2 -4`	7	9–9		10	32
`Bne R1 R0 -12`	7	10–10			32
`Addi R1 R1 -4`	8	11–11		12	32
`Addi R2 R2 -4`	9	12–12		13	33
`Bne R1 R0 -12`	10	13–13			33
`Add.d F20 F2 F2`	15	16–19		20	33
Squashed (last column: cycle squashed)					
`Add.d F20 F2 F2`	3				5
`Bne R1 R0 -12`	13				13
`Addi R2 R2 -4`	12				13
`Addi R1 R1 -4`	11				13

Width 4: 34 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	1	3–3	8–12	14	25
`Add.d F6 F4 F6`	1	25–28		29	30
`Sd F6 0(R2)`	2	4–4	30–34		30
`Addi R1 R1 -4`	2	3–3		4	30
`Addi R2 R2 -4`	2	4–4		5	30
`Bne R1 R0 -12`	2	5–5			31
`Addi R1 R1 -4`	7	8–8		9	31
`Addi R2 R2 -4`	7	9–9		10	31
`Bne R1 R0 -12`	7	10–10			31
`Addi R1 R1 -4`	8	11–11		12	32
`Addi R2 R2 -4`	9	12–12		13	32
`Bne R1 R0 -12`	10	13–13			32
`Add.d F20 F2 F2`	15	16–19		20	32
Squashed (last column: cycle squashed)					
`Add.d F20 F2 F2`	3				5
`Bne R1 R0 -12`	13				13
`Addi R2 R2 -4`	12				13
`Addi R1 R1 -4`	11				13

Table 7: `spec_sample` at widths 1 to 4.

Width 1: 19 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4		5	6
`Ld F4 8(R1)`	2	3–3	4–7	8	9
`Addi R2 R2 -10`	3	4–4		6	10
`Bne R2 R3 -16`	4	7–7			11
`Add.d F1 F2 F3`	9	10–12		13	14
`Ld F4 8(R1)`	10	11–11	12–15	16	17
`Addi R2 R2 -10`	11	12–12		14	18
`Bne R2 R3 -16`	12	15–15			19
Squashed (last column: cycle squashed)					
`Addi R2 R2 -10`	15				15
`Ld F4 8(R1)`	14	15–15			15
`Add.d F1 F2 F3`	13	14			15

Width 2: 15 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4		5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	2	3–3		4	8
`Bne R2 R3 -16`	2	5–5			9
`Add.d F1 F2 F3`	7	8–10		11	12
`Ld F4 8(R1)`	7	8–8	9–12	13	14
`Addi R2 R2 -10`	8	9–9		10	14
`Bne R2 R3 -16`	8	11–11			15
Squashed (last column: cycle squashed)					
`Addi R2 R2 -10`	10				11
`Ld F4 8(R1)`	9	10–10			11
`Add.d F1 F2 F3`	9	10			11

Width 3: 13 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4		5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	1	2–2		3	8
`Bne R2 R3 -16`	2	4–4			8
`Add.d F1 F2 F3`	6	7–9		10	11
`Ld F4 8(R1)`	6	7–7	8–11	12	13
`Addi R2 R2 -10`	6	7–7		8	13
`Bne R2 R3 -16`	7	9–9			13
Squashed (last column: cycle squashed)					
`Addi R2 R2 -10`	8				9
`Ld F4 8(R1)`	8	9–9			9
`Add.d F1 F2 F3`	8				9

Width 4: 13 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F2 F3`	1	2–4		5	6
`Ld F4 8(R1)`	1	2–2	3–6	7	8
`Addi R2 R2 -10`	1	2–2		3	8
`Bne R2 R3 -16`	1	4–4			8
`Add.d F1 F2 F3`	6	7–9		10	11
`Ld F4 8(R1)`	6	7–7	8–11	12	13
`Addi R2 R2 -10`	6	7–7		8	13
`Bne R2 R3 -16`	6	9–9			13
Squashed (last column: cycle squashed)					
`Addi R2 R2 -10`	8				9
`Ld F4 8(R1)`	7	8–8			9
`Add.d F1 F2 F3`	7	8			9

Table 8: `t1_independent` at widths 1 to 4.

Width 1: 20 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F5 0(R1)`	1	2–2	3–6	9	10
`Ld F6 0(R2)`	2	3–3	7–10	13	14
`Add.d F7 F1 F2`	3	4–6		7	15
`Mult.d F8 F3 F4`	4	5–14		15	16
`Add R5 R1 R2`	5	6–6		8	17
`Sub R6 R3 R4`	6	7–7		10	18
`Sub.d F9 F4 F1`	7	8–10		12	19
`Addi R7 R3 100`	8	9–9		11	20

Width 2: 16 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F5 0(R1)`	1	2–2	3–6	9	10
`Ld F6 0(R2)`	1	3–3	7–10	11	12
`Add.d F7 F1 F2`	2	3–5		6	12
`Mult.d F8 F3 F4`	2	3–12		13	14
`Add R5 R1 R2`	3	4–4		5	14
`Sub R6 R3 R4`	3	5–5		7	15
`Sub.d F9 F4 F1`	4	5–7		10	15
`Addi R7 R3 100`	4	6–6		8	16

Width 3: 15 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F5 0(R1)`	1	2–2	3–6	9	10
`Ld F6 0(R2)`	1	3–3	7–10	11	12
`Add.d F7 F1 F2`	1	2–4		5	12
`Mult.d F8 F3 F4`	2	3–12		13	14
`Add R5 R1 R2`	2	3–3		4	14
`Sub R6 R3 R4`	2	4–4		6	14
`Sub.d F9 F4 F1`	3	4–6		8	15
`Addi R7 R3 100`	3	5–5		7	15

Width 4: 14 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F5 0(R1)`	1	2–2	3–6	9	10
`Ld F6 0(R2)`	1	3–3	7–10	11	12
`Add.d F7 F1 F2`	1	2–4		5	12
`Mult.d F8 F3 F4`	1	2–11		12	13
`Add R5 R1 R2`	2	3–3		4	13
`Sub R6 R3 R4`	2	4–4		6	13
`Sub.d F9 F4 F1`	2	3–5		7	13
`Addi R7 R3 100`	2	5–5		8	14

Table 9: `t2_raw_chain` at widths 1 to 4.

Width 1: 26 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F1 F2`	1	2–4		5	6
`Add.d F1 F1 F2`	2	6–8		9	10
`Add.d F1 F1 F2`	3	10–12		13	14
`Add.d F1 F1 F2`	4	14–16		17	18
`Add.d F1 F1 F2`	5	18–20		21	22
`Add.d F1 F1 F2`	6	22–24		25	26

Width 2: 26 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F1 F2`	1	2–4		5	6
`Add.d F1 F1 F2`	1	6–8		9	10
`Add.d F1 F1 F2`	2	10–12		13	14
`Add.d F1 F1 F2`	2	14–16		17	18
`Add.d F1 F1 F2`	3	18–20		21	22
`Add.d F1 F1 F2`	3	22–24		25	26

Width 3: 26 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F1 F2`	1	2–4		5	6
`Add.d F1 F1 F2`	1	6–8		9	10
`Add.d F1 F1 F2`	1	10–12		13	14
`Add.d F1 F1 F2`	2	14–16		17	18
`Add.d F1 F1 F2`	2	18–20		21	22
`Add.d F1 F1 F2`	2	22–24		25	26

Width 4: 26 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1 F1 F2`	1	2–4		5	6
`Add.d F1 F1 F2`	1	6–8		9	10
`Add.d F1 F1 F2`	1	10–12		13	14
`Add.d F1 F1 F2`	1	14–16		17	18
`Add.d F1 F1 F2`	2	18–20		21	22
`Add.d F1 F1 F2`	2	22–24		25	26

Table 10: `t3_structural` at widths 1 to 4.

Width 1: 22 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–11		12	13
`Mult.d F4 F1 F2`	3	4–13		14	15
`Mult.d F5 F1 F2`	5	6–15		16	17
`Add.d F6 F1 F2`	6	7–9		10	18
`Add.d F7 F1 F2`	13	14–16		17	19
`Addi R1 R0 5`	15	16–16		18	20
`Addi R2 R0 6`	17	18–18		19	21
`Addi R3 R0 7`	18	19–19		20	22

Width 2: 21 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–11		12	13
`Mult.d F4 F1 F2`	3	4–13		14	15
`Mult.d F5 F1 F2`	5	6–15		16	17
`Add.d F6 F1 F2`	5	6–8		9	17
`Add.d F7 F1 F2`	13	14–16		17	18
`Addi R1 R0 5`	15	16–16		18	19
`Addi R2 R0 6`	17	18–18		19	20
`Addi R3 R0 7`	17	19–19		20	21

Width 3: 21 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–11		12	13
`Mult.d F4 F1 F2`	3	4–13		14	15
`Mult.d F5 F1 F2`	5	6–15		16	17
`Add.d F6 F1 F2`	5	6–8		9	17
`Add.d F7 F1 F2`	13	14–16		17	18
`Addi R1 R0 5`	15	16–16		18	19
`Addi R2 R0 6`	17	18–18		19	20
`Addi R3 R0 7`	17	19–19		20	21

Width 4: 21 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–11		12	13
`Mult.d F4 F1 F2`	3	4–13		14	15
`Mult.d F5 F1 F2`	5	6–15		16	17
`Add.d F6 F1 F2`	5	6–8		9	17
`Add.d F7 F1 F2`	13	14–16		17	18
`Addi R1 R0 5`	15	16–16		18	19
`Addi R2 R0 6`	17	18–18		19	20
`Addi R3 R0 7`	17	19–19		20	21

Table 11: `t4_forwarding` at widths 1 to 4.

Width 1: 21 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F3 F2 F1`	2	8–10		11	12
`Sd F3 0(R1)`	3	4–4	13–16		13
`Ld F4 0(R2)`	4	5–5	12	14	15
`Ld F5 8(R1)`	5	6–6	7–10	12	16
`Addi R3 R1 8`	6	7–7		8	17
`Sd F1 0(R3)`	7	9–9	18–21		18
`Ld F6 0(R3)`	8	10–10	11	13	19
`Ld F7 0(R1)`	9	11–11	13	15	20

Width 2: 19 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F3 F2 F1`	1	8–10		11	12
`Sd F3 0(R1)`	2	3–3	12–15		12
`Ld F4 0(R2)`	2	4–4	12	13	14
`Ld F5 8(R1)`	3	5–5	7–10	12	14
`Addi R3 R1 8`	3	4–4		5	15
`Sd F1 0(R3)`	4	6–6	16–19		16
`Ld F6 0(R3)`	4	7–7	8	9	16
`Ld F7 0(R1)`	7	8–8	13	14	17

Width 3: 19 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F3 F2 F1`	1	8–10		11	12
`Sd F3 0(R1)`	1	3–3	12–15		12
`Ld F4 0(R2)`	2	4–4	12	13	14
`Ld F5 8(R1)`	2	5–5	7–10	12	14
`Addi R3 R1 8`	2	3–3		4	14
`Sd F1 0(R3)`	3	6–6	16–19		16
`Ld F6 0(R3)`	3	7–7	8	9	16
`Ld F7 0(R1)`	7	8–8	13	14	16

Width 4: 19 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F3 F2 F1`	1	8–10		11	12
`Sd F3 0(R1)`	1	3–3	12–15		12
`Ld F4 0(R2)`	1	4–4	12	13	14
`Ld F5 8(R1)`	2	5–5	7–10	12	14
`Addi R3 R1 8`	2	3–3		4	14
`Sd F1 0(R3)`	2	6–6	16–19		16
`Ld F6 0(R3)`	2	7–7	8	9	16
`Ld F7 0(R1)`	7	8–8	13	14	16

Table 12: `t5_loop` at widths 1 to 4.

Width 1: 34 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F4 F4 F2`	2	8–10		11	12
`Addi R1 R1 4`	3	4–4		5	13
`Bne R1 R3 -16`	4	6–6			14
`Ld F2 0(R1)`	8	9–9	10–13	14	15
`Add.d F4 F4 F2`	9	15–17		18	19
`Addi R1 R1 4`	10	11–11		12	20
`Bne R1 R3 -16`	11	13–13			21
`Ld F2 0(R1)`	12	13–13	14–17	19	22
`Add.d F4 F4 F2`	13	20–22		23	24
`Addi R1 R1 4`	14	15–15		16	25
`Bne R1 R3 -16`	15	17–17			26
`Ld F2 0(R1)`	16	17–17	18–21	22	27
`Add.d F4 F4 F2`	17	24–26		27	28
`Addi R1 R1 4`	18	19–19		20	29
`Bne R1 R3 -16`	19	21–21			30
`Sd F4 0(R3)`	23	24–24	31–34		31
Squashed (last column: cycle squashed)					
`Sd F4 0(R3)`	5	6–6			6
`Add.d F4 F4 F2`	21				21
`Ld F2 0(R1)`	20	21–21			21

Width 2: 31 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F4 F4 F2`	1	8–10		11	12
`Addi R1 R1 4`	2	3–3		4	12
`Bne R1 R3 -16`	2	5–5			13
`Ld F2 0(R1)`	7	8–8	9–12	14	15
`Add.d F4 F4 F2`	7	15–17		18	19
`Addi R1 R1 4`	8	9–9		10	19
`Bne R1 R3 -16`	8	11–11			20
`Ld F2 0(R1)`	9	11–11	13–16	17	20
`Add.d F4 F4 F2`	9	19–21		22	23
`Addi R1 R1 4`	10	12–12		13	23
`Bne R1 R3 -16`	10	14–14			24
`Ld F2 0(R1)`	11	14–14	17–20	21	24
`Add.d F4 F4 F2`	11	23–25		26	27
`Addi R1 R1 4`	12	15–15		16	27
`Bne R1 R3 -16`	12	17–17			28
`Sd F4 0(R3)`	19	20–20	28–31		28
Squashed (last column: cycle squashed)					
`Sd F4 0(R3)`	3	4–4			5
`Bne R1 R3 -16`	17				17
`Addi R1 R1 4`	16				17
`Add.d F4 F4 F2`	16				17
`Ld F2 0(R1)`	13	17–17			17

Width 3: 30 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F4 F4 F2`	1	8–10		11	12
`Addi R1 R1 4`	1	2–2		3	12
`Bne R1 R3 -16`	2	4–4			12
`Ld F2 0(R1)`	6	7–7	8–11	13	14
`Add.d F4 F4 F2`	6	14–16		17	18
`Addi R1 R1 4`	6	7–7		8	18
`Bne R1 R3 -16`	7	9–9			18
`Ld F2 0(R1)`	8	9–9	12–15	16	19
`Add.d F4 F4 F2`	8	18–20		21	22
`Addi R1 R1 4`	8	10–10		12	22
`Bne R1 R3 -16`	9	13–13			22
`Ld F2 0(R1)`	10	13–13	16–19	20	23
`Add.d F4 F4 F2`	10	22–24		25	26
`Addi R1 R1 4`	10	14–14		15	26
`Bne R1 R3 -16`	11	16–16			26
`Sd F4 0(R3)`	18	19–19	27–30		27
Squashed (last column: cycle squashed)					
`Sd F4 0(R3)`	2	3–3			4
`Ld F2 0(R1)`	16				16
`Bne R1 R3 -16`	15				16
`Addi R1 R1 4`	15				16
`Add.d F4 F4 F2`	15				16
`Ld F2 0(R1)`	12	16–16			16

Width 4: 29 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–6	7	8
`Add.d F4 F4 F2`	1	8–10		11	12
`Addi R1 R1 4`	1	2–2		3	12
`Bne R1 R3 -16`	1	4–4			12
`Ld F2 0(R1)`	6	7–7	8–11	13	14
`Add.d F4 F4 F2`	6	14–16		17	18
`Addi R1 R1 4`	6	7–7		8	18
`Bne R1 R3 -16`	6	9–9			18
`Ld F2 0(R1)`	7	9–9	12–15	16	18
`Add.d F4 F4 F2`	7	18–20		21	22
`Addi R1 R1 4`	7	10–10		12	22
`Bne R1 R3 -16`	7	13–13			22
`Ld F2 0(R1)`	8	13–13	16–19	20	22
`Add.d F4 F4 F2`	9	22–24		25	26
`Addi R1 R1 4`	9	14–14		15	26
`Bne R1 R3 -16`	10	16–16			26
`Sd F4 0(R3)`	18	19–19	26–29		26
Squashed (last column: cycle squashed)					
`Sd F4 0(R3)`	2	3–3			4
`Ld F2 0(R1)`	16				16
`Bne R1 R3 -16`	15				16
`Addi R1 R1 4`	15				16
`Add.d F4 F4 F2`	15				16
`Ld F2 0(R1)`	11	16–16			16

Table 13: `t6_beq_mispredict` at widths 1 to 4.

Width 1: 21 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Beq R1 R2 4`	1	2–2			3
`Sub R5 R3 R1`	4	5–5		6	7
`Sub.d F3 F2 F1`	5	6–8		9	10
`Mult.d F4 F1 F2`	6	7–16		17	18
`Beq R1 R3 4`	7	8–8			19
`Addi R6 R0 1`	8	9–9		10	20
`Add R7 R5 R6`	9	11–11		12	21
Squashed (last column: cycle squashed)					
`Addi R4 R0 99`	2				2

Width 2: 18 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Beq R1 R2 4`	1	2–2			3
`Sub R5 R3 R1`	4	5–5		6	7
`Sub.d F3 F2 F1`	4	5–7		8	9
`Mult.d F4 F1 F2`	5	6–15		16	17
`Beq R1 R3 4`	5	6–6			17
`Addi R6 R0 1`	6	7–7		9	18
`Add R7 R5 R6`	6	10–10		11	18
Squashed (last column: cycle squashed)					
`Sub.d F3 F2 F1`	2				2
`Sub R5 R3 R1`	2				2
`Addi R4 R0 99`	1				2

Width 3: 17 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Beq R1 R2 4`	1	2–2			3
`Sub R5 R3 R1`	4	5–5		6	7
`Sub.d F3 F2 F1`	4	5–7		8	9
`Mult.d F4 F1 F2`	4	5–14		15	16
`Beq R1 R3 4`	5	6–6			16
`Addi R6 R0 1`	5	7–7		9	16
`Add R7 R5 R6`	5	10–10		11	17
Squashed (last column: cycle squashed)					
`Beq R1 R3 4`	2				2
`Mult.d F4 F1 F2`	2				2
`Sub.d F3 F2 F1`	2				2
`Sub R5 R3 R1`	1				2
`Addi R4 R0 99`	1				2

Width 4: 16 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Beq R1 R2 4`	1	2–2			3
`Sub R5 R3 R1`	4	5–5		6	7
`Sub.d F3 F2 F1`	4	5–7		8	9
`Mult.d F4 F1 F2`	4	5–14		15	16
`Beq R1 R3 4`	4	6–6			16
`Addi R6 R0 1`	5	7–7		9	16
`Add R7 R5 R6`	5	10–10		11	16
Squashed (last column: cycle squashed)					
`Beq R1 R3 4`	2				2
`Mult.d F4 F1 F2`	2				2
`Sub.d F3 F2 F1`	1				2
`Sub R5 R3 R1`	1				2
`Addi R4 R0 99`	1				2